

本番PC1台における「厳格な論理分離」決定版マニュアル

1. 2つの世界のガチガチな住み分け

項目	現場用（本番環境）	pm_dev（開発・検証環境）
物理PC	本番PC（1台の筐体を共有）	同左
OSユーザー	同一OSユーザーで常時運用（Docker停止防止のためログオフ・ユーザー切替をしない）	同左（同一ユーザー内で論理分離）
フロント (Vue)	Docker内（ポート 8501）	生環境（ポート 8502 / --host 0.0.0.0）
バック (Django)	Docker内（ポート 8081）	生環境（ポート 8082 / 0.0.0.0 起動）
ALLOWED_HOSTS	['本番PCの固定IP', '現場端末アクセス元']	['本番PCの固定IP', 'テスト端末アクセス元']
MySQL	Dockerコンテナ内MySQL（pm_prod_db / 本番用データ、Adminerで操作）	Dockerコンテナ内MySQL（pm_dev_db / テスト用データ、本番MySQLとは別コンテナ）
DBアクセス権限	本番専用DBユーザー（pm_prod_db のみ権限）	開発専用DBユーザー（pm_prod_db への権限は一切付与しない）
Git管理方法	settings.py は共通化してGit管理。差分は .env で管理し、.env のみGit除外	同左

補足:

- ALLOWED_HOSTS=['*'] は禁止。
- 端末IP固定が難しい場合は、固定IPに加えて管理サブネット制限とリバースプロキシ制限を併用する。
- 本番PCで Docker Desktop を使う場合、OSユーザー切替やログオフで本番コンテナが停止する可能性があるため、運用中は同一ユーザーで固定する。
- DBの役割分担は「本番=Docker内MySQL（本番コンテナ）」「開発=Docker内MySQL（開発コンテナ）」とし、共用しない。ローカルMySQLを別途インストールするよりDocker内で分離した方が管理が楽。

2. スマホ・現場端末テスト時の正しい通信・権限ルート

外部端末から pm_dev（実験室）を安全に叩くための、論理分離された通信フロー。

```
[テスト用スマホ]（固定IPまたは許可サブネット）
|
v ① フロントへアクセス（http://本番PCのIP:8502）
[本番PC 生環境：開発用ディレクトリの Vue.js]
|
```

```
v ② API通信 (VITE_API_BASE_URL=http://本番PCのIP:8082/api/)
[本番PC 生環境: 開発用ディレクトリの Django]
|
v ③ 開発専用DBユーザーで開発用MySQLコンテナに接続
[ Docker: 開発用MySQLコンテナ (pm_dev_db のみ権限)]
```

防御ポイント:

- 本番DBと開発DBはDocker内の別コンテナとして分離する（ホスト名・ポートが異なる）。
- 開発側には本番DBの接続情報（ホスト・ユーザー・パスワード）を保持させない。
- 本番アプリ接続先は `DB_HOST=mysql`（本番用コンテナ）、開発アプリ接続先は `DB_HOST=mysql-dev`（開発用コンテナ）に固定する。
- これにより本番DBへの誤接続・誤更新リスクを大幅に低減する。

2.1 DB構成の矛盾防止ルール（必須）

1. `.env.prod` と `.env.dev` で接続先・ユーザー・パスワードを完全分離し、共用しない。
2. Adminerは本番操作用途に限定し、接続先ラベルを明示して誤接続を防ぐ。
3. 開発起動前チェックで `ENV=dev` かつ `DB_HOST=mysql-dev` を強制検証する。
4. 本番バックアップ（Docker内MySQL）と開発バックアップ（ローカルMySQL）はジョブと保存先を分離する。

3. ラインを止めないための YOLOv8 運用5大鉄則

1. 時間帯制限（学習の禁止） 開発側の重い学習（`train`）は、生産稼働時間帯は実施しない。稼働時間帯は録画動画による推論（`predict`）のみとする。
2. GPUメモリ上限設定 開発側PyTorchのGPU割り当て上限を設定し、本番コンテナのGPU使用を圧迫しない。
3. 同一ユーザー内での環境分離 作業ディレクトリ、`venv`、ログ保存先、環境変数（`.env.prod` / `.env.dev`）を完全分離する。
4. 温度・負荷監視と自動停止 NVML等で温度/負荷を監視し、閾値超過時は本番保護のため開発側YOLOプロセスを自動停止する。
5. バックアップ分離 本番DBバックアップと開発DBバックアップを別ジョブ・別保存先で運用し、相互上書きを防ぐ。

4. 追加の必須ガード

1. 本番コンテナ操作コマンドと開発起動コマンドを分離し、エイリアス/スクリプトで誤操作防止
2. 開発プロセスには本番用 `.env` を読み込ませない（`ENV=dev` を強制）
3. 接続前チェックをスクリプト化（DB名が `pm_prod_db` / `pm_dev_db` のどちらかを必ず検証）
4. 開発側Djangoの `runserver` は検証用途限定。継続公開が必要なら `gunicorn` 等へ切替
5. 運用中のOSユーザー切替・ログオフ禁止（本番Docker停止防止）

5. 結論

この設計は「動くかどうか」ではなく、開発中のバグや高負荷が起きた場合でも、本番への実害を最小化するための運用設計である。

表現上は「完全分離」ではなく、単一筐体内での「厳格な論理分離」と定義する。

6. マニュアルを具現化する「論理分離」実装手順書

6.1 MySQL: 開発用ユーザーの作成と権限縛り

PCローカルMySQLに `root` で接続し、以下を実行する。

```
CREATE DATABASE pm_dev_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

CREATE USER 'pm_dev_user'@'127.0.0.1' IDENTIFIED BY '開発用パスワード';
CREATE USER 'pm_dev_user'@'localhost' IDENTIFIED BY '開発用パスワード';

GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, ALTER, INDEX ON pm_dev_db.* TO
'pm_dev_user'@'127.0.0.1';
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, ALTER, INDEX ON pm_dev_db.* TO
'pm_dev_user'@'localhost';

FLUSH PRIVILEGES;
```

運用メモ:

- `DROP` 権限は通常付与しない（必要時のみ一時付与）。
- 本番Docker MySQLにはこのユーザーを作成しない。

6.2 Django: `.env` 分離と起動前チェック

開発用 `.env.dev` (Git除外) 例:

```
ENV=dev
DB_NAME=pm_dev_db
DB_USER=pm_dev_user
DB_PASSWORD=開発用パスワード
DB_HOST=127.0.0.1
DB_PORT=3306
ALLOWED_HOSTS=127.0.0.1,本番PC固定IP
```

`settings.py` ガード例:

```
import os
from django.core.exceptions import ImproperlyConfigured

ENV = os.getenv("ENV", "prod")
if ENV == "dev":
    from dotenv import load_dotenv
```

```
load_dotenv()

DB_HOST = os.getenv("DB_HOST")
DB_NAME = os.getenv("DB_NAME")

if ENV == "dev":
    if DB_HOST != "127.0.0.1" or DB_NAME == "pm_prod_db":
        raise ImproperlyConfigured("危険: dev設定で本番向け接続が検出されました。")
else:
    if DB_HOST == "127.0.0.1":
        raise ImproperlyConfigured("危険: prod設定でローカルDB接続が検出されまし
た。")
```

6.3 YOLOv8: GPU上限と温度の継続監視

```
import sys
import time
import torch
from pynvml import nvmlInit, nvmlDeviceGetHandleByIndex, nvmlDeviceGetTemperature,
NVML_TEMPERATURE_GPU

def init_gpu_guard():
    if torch.cuda.is_available():
        torch.cuda.set_per_process_memory_fraction(0.4, device=0)
    nvmlInit()
    return nvmlDeviceGetHandleByIndex(0)

def ensure_safe_temperature(handle, limit=80):
    temperature = nvmlDeviceGetTemperature(handle, NVML_TEMPERATURE_GPU)
    if temperature > limit:
        print(f"警告: GPU温度 {temperature}C。開発プロセスを停止します。")
        sys.exit(1)

handle = init_gpu_guard()

# 推論ループ内で定期監視（例: 30秒間隔）
last_check = 0
while True:
    now = time.time()
    if now - last_check >= 30:
        ensure_safe_temperature(handle)
        last_check = now
    # ここで推論処理を実行
    # ...
    break
```

6.4 誤操作防止: 開発起動専用エイリアス

PowerShellプロファイルに登録する。

```
function Start-PmDev {  
    Set-Location "C:\factory\pm_dev"  
    $env:ENV = "dev"  
    $env:DJANGO_SETTINGS_MODULE = "project.settings"  
    python manage.py runserver 0.0.0.0:8001  
}  
Set-Alias -Name devrun -Value Start-PmDev
```

運用メモ:

- 本番Docker操作コマンド（`docker compose down` 等）は別名コマンドに分離する。
- 断定ではなく「リスクを大幅に低減する」という表現で運用文書を統一する。